HTML5 Exercises

Task 1 — Create the HTML5 Base Template

index.html — Base Document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>   <meta name="description" content="Local Community Event Portal" />
<title>Community Event Portal</title>

  <style>
    /* Reset and base */     * { margin: 0; padding: 0;
box-sizing: border-box; }
    body { font-family: Arial, sans-serif; background: #f5f5f5; color: #333; }

    /* Navigation */
    nav { background: #1a3c5e; padding: 12px 24px; }
    nav a { color: #fff; text-decoration: none; margin-right: 20px; font-weight: bold; }
nav a:hover { text-decoration: underline; }

    /* Header */
    header { background: #2a6496; color: white; padding: 30px 24px; }

    /* Main */
    main { max-width: 960px; margin: 24px auto; padding: 0 16px; }

    /* Footer */
    footer { background: #1a3c5e; color: #ccc; text-align: center; padding: 16px; }
</style>
</head>
<body>

  <!-- Navigation -->
  <nav>
    <a href="#home">Home</a>
    <a href="#events">Events</a>
<a href="#contact">Contact</a>
  </nav>

  <!-- Header / Hero -->
  <header id="home">
    <h1>Welcome to the Community Event Portal</h1>     <p>Your one-stop
destination for local city events and services.</p>
  </header>

  <!-- Main Content -->
  <main>
    <section id="events">
      <h2>Upcoming Events</h2>       <p>Browse and
register for events in your area.</p>
    </section>
    <section id="contact">
      <h2>Contact Us</h2>       <p>Reach out to the city
council for more information.</p>
    </section>
  </main>

  <!-- Footer -->
  <footer>     <p>&copy; 2024 City Council Community Portal. All rights
reserved.</p>
  </footer>
```

```
</body>
</html>
```

Chrome DevTools — Elements panel:
  html head meta
charset="UTF-8" meta
name="viewport" ... title:
Community Event Portal style
body nav header#home
main section#events
section#contact footer

Browser output:
  Dark blue navbar with Home / Events / Contact links Blue
header banner: "Welcome to the Community Event Portal"
  Two content sections below
  Dark footer with copyright

Task 2 — Navigation and Linking

nav section with anchor links and external link

```
<!-- Navigation with in-page anchor links -->
<nav>   <!-- In-page smooth-scroll
links -->
  <a href="#home">Home</a>
  <a href="#events">Events</a>
  <a href="#contact">Contact</a>

  <!-- External link opens in a new tab -->   <a href="help.html"
target="_blank" rel="noopener noreferrer">Help</a>
</nav>

<!-- Sections with matching IDs for anchor targets -->
<main>

  <section id="home">
    <h2>Home</h2>     <p>Welcome to the portal. Use the links
above to navigate.</p>
  </section>

  <section id="events">
    <h2>Events</h2>     <p>Find and register for upcoming
community events here.</p>
    <!-- In-section back-to-top link -->
<a href="#home">&#8593; Back to Top</a>
  </section>

  <section id="contact">
    <h2>Contact</h2>
    <address>
      City Council Office<br />
      123 Main Street, Springfield<br />       <a
href="mailto:council@city.gov">council@city.gov</a><br />
<a href="tel:+911234567890">+91 12345 67890</a>
    </address>     <a href="#home">&#8593;
Back to Top</a>
  </section>

</main>
```

rel="noopener noreferrer" is a security best practice on target="_blank" links — prevents the new tab from accessing the opener page via window.opener.

```
Browser output:   Clicking "Events" in the navbar scrolls the page to
the #events section
  Clicking "Help" opens help.html in a new browser tab
  "Back to Top" arrow links return focus to the top of the page   Contact
section shows formatted address with clickable email and phone links

Chrome DevTools — Console:
  No errors.
  Elements panel: href="#events" highlighted when anchor is clicked
```

Task 3 — Welcome Message with Styling and ID/Class

welcomeBanner div with internal CSS, inline style, and .highlight class

```
<style>   /* ID selector — unique
element */   #welcomeBanner {
    background-color: #1a6eb5;    /* blue background
*/     color: white;     padding: 20px 24px;
border-radius: 6px;     margin-bottom: 20px;   }

  /* Class selector — reusable on any element
*/   .highlight {     background-color:
#fff3cd;     padding: 2px 6px;     border-
radius: 3px;     font-weight: bold;     color:
#333;   }
</style>
```

```
<!-- Block element: div with id -->
<div id="welcomeBanner">   <h2>Welcome back, <span
class="highlight">Rahul!</span></h2>
  <p>     You are now logged in to the Community Event
Portal.
    <!-- Inline element: span with inline style -->
    <span style="color: #ffeb3b; font-weight: bold; font-size: 1.1em;">
Special Offer: Register for 2 events and get 1 FREE!
    </span>
  </p>   <p class="highlight">Next event starts in 3 days. Don't miss
it!</p>
</div>

<!-- Block vs Inline explanation in comments:
     <div>  : block-level — takes full width, starts on a new line
<span> : inline — flows within text, only as wide as its content
id     : unique identifier — one element per page      class  :
reusable — can apply to many elements              -->
```

Browser output:

- Blue banner box with white text
- "Rahul!" highlighted in yellow box (class="highlight")
- Special offer text in bold yellow (#ffeb3b)
- "Next event starts in 3 days" paragraph with yellow highlight background
- Inline span flows within the paragraph text without breaking the line

Task 4 — Image Gallery for Community Events

Table layout with 2 rows x 3 columns of images

```
<style>
  .gallery-table { border-collapse: collapse; width: 100%; margin-top: 16px;
}   .gallery-table td { padding: 8px; text-align: center; vertical-align:
top; }   .gallery-table caption {     font-size: 1.2em; font-weight: bold;
padding: 8px; caption-side: top; color: #1a3c5e;   }   .event-img {
    width: 280px; height: 180px; object-fit: cover;     border:
3px solid #2a6496; border-radius: 4px;     display: block; margin:
0 auto;   }   .img-caption { font-size: 0.85em; color: #555;
margin-top: 4px; }
</style>

<section id="gallery">
  <h2>Past Events Gallery</h2>

  <table class="gallery-table">     <caption>Community
Events — 2024 Highlights</caption>

    <!-- Row 1 -->
    <tr>
      <td>         <img src="images/sports-day.jpg"
class="event-img"              alt="Annual Sports Day
2024"              title="Annual Sports Day — March
2024" />         <p class="img-caption">Annual Sports
Day</p>
      </td>
      <td>         <img src="images/cultural-
fest.jpg"              class="event-img"
alt="Cultural Festival 2024"
title="Cultural Festival — April 2024" />         <p
class="img-caption">Cultural Festival</p>
      </td>
      <td>         <img src="images/tree-
planting.jpg"              class="event-img"
alt="Tree Planting Drive"              title="Tree
Planting Drive — May 2024" />         <p class="img-
caption">Tree Planting Drive</p>
      </td>
    </tr>

    <!-- Row 2 -->
    <tr>
```

```
      <td>         <img src="images/health-
camp.jpg"              class="event-img"
alt="Free Health Camp"              title="Free
Health Camp — June 2024" />         <p class="img-
caption">Free Health Camp</p>
      </td>
      <td>         <img src="images/food-fair.jpg"
class="event-img"              alt="Community Food
Fair"              title="Community Food Fair — July
2024" />         <p class="img-caption">Community Food
Fair</p>
      </td>
      <td>
        <img src="images/tech-workshop.jpg"
             class="event-img"
alt="Tech Skills Workshop"
             title="Tech Skills Workshop — August 2024" />
<p class="img-caption">Tech Skills Workshop</p>
      </td>
    </tr>
  </table>
</section>
```

```
Browser output:
  Table caption at top: "Community Events — 2024 Highlights"
  Row 1: Sports Day | Cultural Festival | Tree Planting  (3 images side by side)
Row 2: Health Camp | Food Fair | Tech Workshop          (3 images side by side)
  Each image: 280x180px, blue 3px border, rounded corners
  Caption text in small grey font below each image   Hovering over image shows
title tooltip (e.g. "Annual Sports Day — March 2024")   alt text shown if image
file not found (broken image placeholder)
```

Task 5 — Event Registration Form

Registration form with validation, output element, and CSS styling

```
<style>   .reg-form { background: white; padding: 28px; border-radius:
8px;              box-shadow: 0 2px 8px rgba(0,0,0,.1); max-width:
520px; }   .reg-form h3 { color: #1a3c5e; margin-bottom: 18px; }
  .form-group { margin-bottom: 14px; }   .form-group label { display: block;
font-weight: bold; margin-bottom: 4px; }
  .form-group input,
  .form-group select,   .form-
group textarea {
    width: 100%; padding: 8px 10px; border: 1px solid
#ccc;     border-radius: 4px; font-size: 0.95em;   }
  .form-group input:focus,   .form-group textarea:focus { border-
color: #2a6496; outline: none; }   .btn-submit {
    background: #2a6496; color: white; border:
none;     padding: 10px 24px; border-radius: 4px;
cursor: pointer; font-size: 1em; width: 100%;   }
  .btn-submit:hover { background: #1a3c5e; }
#confirmation { color: green; font-weight: bold;
margin-top: 12px; display: none; } </style>

<section id="register">
  <h2>Event Registration</h2>

  <form class="reg-form" id="regForm"
onsubmit="handleSubmit(event)">     <h3>Register
for an Event</h3>

    <!-- Name -->
    <div class="form-group">
      <label for="fullName">Full Name *</label>
<input type="text" id="fullName" name="fullName"
placeholder="Enter your full name"
autofocus required />     </div>

    <!-- Email -->
    <div class="form-group">
      <label for="email">Email Address *</label>
<input type="email" id="email" name="email"
placeholder="you@example.com"
required />     </div>

    <!-- Date -->
    <div class="form-group">
      <label for="eventDate">Preferred Date *</label>       <input
type="date" id="eventDate" name="eventDate" required />
    </div>

    <!-- Event Type -->
    <div class="form-group">
      <label for="eventType">Event Type *</label>
      <select id="eventType" name="eventType" required>
        <option value="">-- Select an event --</option>
        <option value="sports">Sports Day</option>
        <option value="cultural">Cultural Festival</option>
        <option value="health">Health Camp</option>
<option value="tech">Tech Workshop</option>
      </select>
    </div>
    <!-- Message -->
    <div class="form-group">
      <label for="message">Message / Special Requirements</label>
<textarea id="message" name="message" rows="4"
               placeholder="Any special requirements or questions...">
</textarea>
    </div>

    <button type="submit" class="btn-submit">Register Now</button>
```

```
    <!-- output element shows confirmation message after submit -->
    <output id="confirmation" name="confirmation" for="regForm">
Thank you! Your registration has been submitted successfully.
    </output>
  </form>
</section>

<script>
  function handleSubmit(e) {
    e.preventDefault();
    const name  = document.getElementById('fullName').value;
const event = document.getElementById('eventType').value;
const conf  = document.getElementById('confirmation');
conf.textContent =       'Thank you, ' + name + '! You are
registered for: ' + event;     conf.style.display = 'block';   }
</script>
```

```
Browser output:
  White card form with shadow, max-width 520px   "Full Name" field is
auto-focused on page load (cursor blinks inside it)
  All fields have placeholder text as hints
  Submitting with empty required field: browser shows built-in red tooltip
"Please fill in this field" / "Please include an @ in the email address"
  Date picker opens calendar widget on click
  Dropdown shows 4 event options

On valid submit:
  Green message appears below button:
  "Thank you, Rahul! You are registered for: sports"
  (output element content updated by handleSubmit function)
```

Task 6 — Event Feedback with Events Handling

onblur, onchange, onclick, ondblclick, keyboard events

```
<style>   .feedback-form { background: white; padding: 24px; border-radius:
8px;                 max-width: 520px; box-shadow: 0 2px 8px rgba(0,0,0,.1);
}   .field-error  { color: red; font-size: 0.85em; display: none; }
  .field-ok     { color: green; font-size: 0.85em; display: none; }
#feeDisplay   { color: #2a6496; font-weight: bold; margin-top: 6px; }
  #charCount    { font-size: 0.8em; color: #777; }   #eventImg     {
width: 200px; cursor: pointer; border: 2px solid #ccc;
border-radius: 4px; transition: width 0.3s ease; }   #eventImg.enlarged
{ width: 380px; }
</style>

<section id="feedback">
  <h2>Event Feedback</h2>
  <div class="feedback-form">

    <!-- 1. onblur: validate phone number when user leaves the field -->
    <div class="form-group">
      <label for="phone">Phone Number</label>
<input type="tel" id="phone" name="phone"
placeholder="10-digit number"
onblur="validatePhone()" />       <span
class="field-error" id="phoneError">
Please enter a valid 10-digit phone number.
      </span>       <span class="field-ok" id="phoneOk">Phone number
looks good!</span>
    </div>

    <!-- 2. onchange: show event fee when dropdown selection changes -->
    <div class="form-group">
      <label for="feedbackEvent">Select Event</label>
      <select id="feedbackEvent" onchange="showFee(this.value)">
        <option value="">-- Choose event --</option>
        <option value="sports">Sports Day</option>
        <option value="cultural">Cultural Festival</option>
        <option value="health">Health Camp</option>
<option value="tech">Tech Workshop</option>
```

```
      </select>       <p
id="feeDisplay"></p>
    </div>

    <!-- 3. ondblclick: double-click image to enlarge -->
    <div class="form-group">
      <label>Double-click image to enlarge</label><br
/>       <img id="eventImg" src="images/sports-day.jpg"
alt="Community Event"
ondblclick="toggleEnlarge()" />     </div>

    <!-- 4. Keyboard events: count characters in textarea -->
    <div class="form-group">
      <label for="feedbackText">Your
Feedback</label>       <textarea id="feedbackText"
rows="4"                 placeholder="Share your
thoughts..."                 onkeyup="countChars()"
onkeydown="checkKey(event)">       </textarea>
<span id="charCount">Characters typed: 0</span>
    </div>

    <!-- 5. onclick: show confirmation -->
<button onclick="submitFeedback()"
            style="background:#2a6496;color:white;border:none;
padding:10px 20px;border-radius:4px;cursor:pointer;">       Submit
Feedback
    </button>     <p id="feedbackConfirm" style="color:green;font-
weight:bold;display:none;"></p>
  </div>
</section>

<script>   // 1. onblur — validate
phone number   function
validatePhone() {
    const phone  =
document.getElementById('phone').value.trim();     const err
= document.getElementById('phoneError');     const ok     =
document.getElementById('phoneOk');     const isValid = /^[0-
9]{10}$/.test(phone);     err.style.display = isValid ? 'none'
: 'block';     ok.style.display  = isValid ? 'block' : 'none';
}

  // 2. onchange — show registration
fee   function showFee(eventType) {
const fees = {
      sports:   'Registration Fee: FREE',
cultural: 'Registration Fee: Rs 50',
health:   'Registration Fee: FREE',
tech:     'Registration Fee: Rs 100'     };
    document.getElementById('feeDisplay').textContent
=       fees[eventType] || '';   }

  // 3. ondblclick — toggle image size
function toggleEnlarge() {
    document.getElementById('eventImg').classList.toggle('enlarged');
}

  // 4. onkeyup — count characters
function countChars() {
    const len =
document.getElementById('feedbackText').value.length;
document.getElementById('charCount').textContent =       'Characters
typed: ' + len;
  }

  // 4b. onkeydown — detect Enter
key   function checkKey(e) {     if
(e.key === 'Enter') {
      console.log('Enter pressed in feedback textarea');
}
  }
```

```
  // 5. onclick — submit feedback
function submitFeedback() {
    const text =
document.getElementById('feedbackText').value.trim();     const conf
= document.getElementById('feedbackConfirm');     if (!text) {
      alert('Please enter your feedback before
submitting.');       return;     }
    conf.textContent = 'Thank you for your
feedback!';     conf.style.display = 'block';   }
</script> Browser events
```

demo output:

onblur (phone field):

Typing "9876543210" and clicking away → "Phone number looks good!" in green

Typing "12345" and clicking away → "Please enter a valid 10-digit phone number." in red

onchange (dropdown):

Selecting "Tech Workshop" → "Registration Fee: Rs 100" appears below select

Selecting "Sports Day" → "Registration Fee: FREE" appears

ondblclick (image): Single click: nothing changes Double click: image smoothly expands from 200px to 380px (CSS transition) Double click again: shrinks back to 200px

onkeyup (textarea):

Typing "Hello" → "Characters typed: 5" shown below textarea
Each keystroke updates the counter in real time

onclick (button):

Clicking with empty textarea → alert "Please enter your feedback..."

Clicking with text → "Thank you for your feedback!" in green

Task 7 — Video Invite with Media Events

video element with oncanplay, onbeforeunload

```
<style>   .video-section { background: white; padding: 24px; border-radius:
8px;                max-width: 640px; box-shadow: 0 2px 8px rgba(0,0,0,.1);
}   #videoStatus  { margin-top: 10px; font-style: italic; color: #2a6496; }
video          { width: 100%; border-radius: 4px; border: 2px solid #ddd; }
</style>

<section id="video-invite">
  <h2>Event Promo Video</h2>
  <div class="video-section">
    <p>Watch the invitation video for our upcoming Community
Festival:</p>

    <!-- HTML5 video element with multiple sources for browser compatibility --
>     <video id="promoVideo"            controls
          poster="images/video-poster.jpg"
oncanplay="videoReady()"
onplay="videoPlaying()"
onpause="videoPaused()"
onended="videoEnded()">

      <!-- Multiple sources: browser picks the first format it supports -->
      <source src="videos/community-event-promo.mp4"  type="video/mp4" />
      <source src="videos/community-event-promo.webm" type="video/webm" />
<!-- Fallback text for browsers that do not support video tag -->
<p>Your browser does not support HTML5 video.          <a
href="videos/community-event-promo.mp4">Download the video</a>.</p>
    </video>

    <p id="videoStatus">Loading video...</p>
  </div>
</section>

<script>   // oncanplay: fires when enough data is loaded to
start playback   function videoReady() {
    document.getElementById('videoStatus').textContent
=       'Video ready to play. Press the play button!';
}

  function videoPlaying() {
    document.getElementById('videoStatus').textContent
=       'Playing...';   }

  function videoPaused() {
    document.getElementById('videoStatus').textContent
=       'Video paused.';   }

  function videoEnded() {
    document.getElementById('videoStatus').textContent =
'Video finished. Thank you for watching!';
  }

  // onbeforeunload: warn user if they try to leave page with unfinished form
let formDirty = false;

  // Mark form as dirty when user types in any input
  document.querySelectorAll('input, textarea, select').forEach(el =>
{     el.addEventListener('input', () => { formDirty = true; });
});

  window.onbeforeunload = function(e) {
    if (formDirty) {
      const msg = 'You have unsaved form data. Are you sure you want to leave?';
      e.returnValue = msg;   // required for
Chrome       return msg;     }
  };
```

```
</script>
```

Browser output:

oncanplay:

Page loads → video poster image shown (thumbnail before play)

Once enough data buffered: "Video ready to play. Press the play button!"
(Status text appears below the video player)

onplay:

Clicking play button → status changes to "Playing..."

onpause:

Clicking pause → status changes to "Video paused."

onended:

Video reaches end → "Video finished. Thank you for watching!"

onbeforeunload: User starts typing in the registration form
(formDirty = true)

User then clicks the browser back button or closes the tab
Chrome shows dialog: "Leave site? Changes you made may not be saved."

Clicking "Stay" keeps the user on the page
Clicking "Leave" navigates away

controls attribute provides: play/pause, seek bar, volume, fullscreen button
poster attribute shows thumbnail image before video is played

Task 8 — Saving User Preferences

localStorage, sessionStorage, and Clear Preferences button

```
<style>   .prefs-section { background: white; padding: 24px; border-radius:
8px;                    max-width: 460px; box-shadow: 0 2px 8px
rgba(0,0,0,.1); }   #prefStatus { margin-top: 8px; color: #2a6496; font-
style: italic; }
</style>

<section id="preferences">
  <h2>User Preferences</h2>
  <div class="prefs-section">     <p>Your preferred event
type is saved across visits.</p>

    <div class="form-group">
      <label for="prefEvent">Preferred Event Type</label>
      <select id="prefEvent" onchange="savePreference(this.value)">
        <option value="">-- Choose your preference --</option>
        <option value="sports">Sports Day</option>
        <option value="cultural">Cultural Festival</option>
        <option value="health">Health Camp</option>
<option value="tech">Tech Workshop</option>
      </select>
    </div>

    <p id="prefStatus"></p>

    <button onclick="clearPreferences()"
            style="background:#c0392b;color:white;border:none;
padding:8px 16px;border-radius:4px;cursor:pointer;">       Clear
Preferences
    </button>
  </div>
</section>

<script>
  // On page load: retrieve saved preference and pre-select it
window.addEventListener('DOMContentLoaded', function () {     const saved =
localStorage.getItem('preferredEvent');     if (saved) {
      const select = document.getElementById('prefEvent');
select.value = saved;
      document.getElementById('prefStatus').textContent =
'Welcome back! Loaded your saved preference: ' + saved;
console.log('Loaded from localStorage:', saved);     }

    // sessionStorage: store session visit count (resets on tab
close)     let visits = sessionStorage.getItem('visitCount') || 0;
visits++;
    sessionStorage.setItem('visitCount', visits);
console.log('Page visits this session:', visits);   });

  //  Save preference when dropdown changes   function savePreference(value) {     if
(!value) return;
    localStorage.setItem('preferredEvent', value);
document.getElementById('prefStatus').textContent =       'Preference
saved: ' + value + ' (persists across browser sessions)';
console.log('Saved to localStorage:', value);   }

  //  Clear all stored preferences   function clearPreferences() {
    localStorage.clear();       // remove all localStorage entries
sessionStorage.clear();     // remove all sessionStorage entries
document.getElementById('prefEvent').value = '';
document.getElementById('prefStatus').textContent =       'All
preferences cleared.';
    console.log('localStorage and sessionStorage cleared.');
}
</script>
```

Browser output — first visit: Dropdown shows "-- Choose your preference --" (nothing pre-selected)

```
  User selects "Tech Workshop"
  Status: "Preference saved: tech (persists across browser sessions)"

  DevTools → Application → Local Storage → http://localhost:
    Key: "preferredEvent"   Value: "tech"

Browser output — after page reload:
  Dropdown auto-selects "Tech Workshop"   Status: "Welcome
back! Loaded your saved preference: tech"
  Console: "Loaded from localStorage: tech"

  DevTools → Application → Session Storage:
    Key: "visitCount"  Value: "2"  (increments each reload, resets on tab close)

Clicking "Clear Preferences":
  Dropdown resets to "-- Choose your preference --"
  Status: "All preferences cleared."
  DevTools → Local Storage: empty
  DevTools → Session Storage: empty
```

Task 9 — Geolocation for Event Mapping

getCurrentPosition with error handling and high accuracy

```
<style>   .geo-section { background: white; padding: 24px; border-radius:
8px;                  max-width: 480px; box-shadow: 0 2px 8px
rgba(0,0,0,.1); }   #geoResult  { margin-top: 12px; padding: 10px;
border-radius: 4px;                 background: #f0f7ff; border-left: 4px
solid #2a6496; }   #geoError   { color: red; margin-top: 8px; display:
none; }
</style>

<section id="geolocation">
  <h2>Find Nearby Events</h2>
  <div class="geo-section">     <p>Click the button to detect your location
and find events near you.</p>

    <button id="geoBtn" onclick="findNearbyEvents()"
style="background:#2a6496;color:white;border:none;
padding:10px 20px;border-radius:4px;cursor:pointer;">       Find Nearby
Events
    </button>

    <div id="geoResult" style="display:none;">
      <strong>Your Location:</strong><br />
      <span id="latDisplay"></span><br />
      <span id="lonDisplay"></span><br />
      <span id="accDisplay"></span><br />
<span id="nearestEvent"></span>
    </div>

    <p id="geoError"></p>
  </div>
</section>

<script>
  function findNearbyEvents() {     //
Check if Geolocation API is supported
if (!navigator.geolocation) {
      showGeoError('Geolocation is not supported by your
browser.');       return;     }

    document.getElementById('geoBtn').textContent = 'Detecting...';
document.getElementById('geoError').style.display = 'none';

    // Geolocation options
const options = {
      enableHighAccuracy: true,   // use GPS if available (more accurate,
slower)       timeout:            10000,  // wait max 10 seconds
maximumAge:         0       // do not use cached position     };

    navigator.geolocation.getCurrentPosition(
onSuccess,   // callback for success
onError,     // callback for error/denial
options     );   }

  function onSuccess(position) {
    const lat = position.coords.latitude.toFixed(6);
const lon = position.coords.longitude.toFixed(6);
const acc = position.coords.accuracy.toFixed(0);

    document.getElementById('latDisplay').textContent = 'Latitude:  ' + lat;
document.getElementById('lonDisplay').textContent = 'Longitude: ' + lon;
document.getElementById('accDisplay').textContent = 'Accuracy:  ' + acc + ' metres';     // Simulate
finding nearest event based on rough location check     let nearest = 'Community Sports Ground (2.3 km
away)';     if (lat > 19) nearest = 'Azad Maidan Event Centre (1.1 km away)';
document.getElementById('nearestEvent').textContent =       'Nearest Event Venue: ' + nearest;

    document.getElementById('geoResult').style.display = 'block';
document.getElementById('geoBtn').textContent = 'Find Nearby Events';
console.log('Position obtained:', position.coords);   }
```

```
  function onError(error) {
let msg = '';     switch
(error.code) {       case
error.PERMISSION_DENIED:
        msg = 'Location access denied. Please allow location permission in browser
settings.';         break;       case error.POSITION_UNAVAILABLE:
        msg = 'Location information is unavailable. Please check your device
settings.';         break;       case error.TIMEOUT:
        msg = 'Request timed out. Please try
again.';         break;       default:
        msg = 'An unknown error occurred (' + error.message + ')';
}
    showGeoError(msg);
    document.getElementById('geoBtn').textContent = 'Find Nearby Events';
}

  function showGeoError(msg) {
    const errEl =
document.getElementById('geoError');
errEl.textContent = msg;     errEl.style.display =
'block';   }
</script>
```

```
Browser output — permission granted:   Chrome shows permission prompt: "community-
portal.com wants to know your location"   Clicking "Allow":
    Blue result box appears:
      Latitude:  18.975100
      Longitude: 72.825800
      Accuracy:  25 metres       Nearest Event Venue: Azad
Maidan Event Centre (1.1 km away)

  Console:     Position obtained: GeolocationCoordinates {latitude:
18.975100, ...}

Browser output — permission denied:
  Clicking "Block" in the prompt:     Red error text: "Location access
denied. Please allow location permission     in browser settings."

Browser output — timeout:
  If GPS takes longer than 10 seconds:
"Request timed out. Please try again."


DevTools → Console:
  Geolocation API available: navigator.geolocation is defined
  enableHighAccuracy: true tries GPS hardware before falling back to Wi-Fi/IP
```

Task 10 — Debugging with Chrome DevTools

Console logs, breakpoints, and live style editing

```
<!-- Debug-friendly script with console logging throughout -->
<script>
  // ■■ 1. Console logging at key points ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
console.log('Portal script loaded.');
  console.log('DOM ready, initialising preferences...');

  window.addEventListener('DOMContentLoaded', function () {
console.group('Initialisation');

    // Log each value as it loads
    const savedPref = localStorage.getItem('preferredEvent');
console.log('Saved preference from localStorage:', savedPref);

    const visitCount = sessionStorage.getItem('visitCount');
console.log('Session visit count:', visitCount);

    console.groupEnd();
});
```

```
  // ■■ 2. Debug function: expose variable state for inspection
■■■■■■■■■■■■■■■■   function debugState() {     const state = {
formDirty:       formDirty,
      savedPreference: localStorage.getItem('preferredEvent'),
sessionVisits:   sessionStorage.getItem('visitCount'),       videoStatus:
document.getElementById('videoStatus')?.textContent     };
    console.table(state);    // displays as a neat table in console     return
state;             // calling debugState() in console shows the object   }

  // ■■ 3. Error boundary example
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■   function
riskyOperation() {     try {
      const el = document.getElementById('nonExistentElement');
      el.textContent = 'This will throw';   // TypeError: Cannot set property of null
} catch (err) {
      console.error('Caught error:', err.message);
      console.warn('Check that the element ID is correct in the HTML.');
}
  }
</script>
```

Chrome DevTools Usage Steps

```
Steps performed in Chrome DevTools:

1. Open DevTools
   Right-click on page → Inspect
   OR press F12 (Windows) / Cmd+Option+I (Mac)

2. Elements Panel — Live Style Editing
   Click on the nav element in the Elements tree    In the Styles pane
(right side), click on the background colour value
   Change #1a3c5e to #e63946    Observe nav bar turns red instantly (page-
only change — not saved to file)    Uncheck a CSS property checkbox to
toggle it off temporarily

3. Console Panel — Run and View Logs
   Click the Console tab
Page loads and prints:
     Portal script loaded.
     Initialisation (group)
       Saved preference from localStorage: tech
       Session visit count: 3
Type directly in console:
     debugState()
     → Shows table with formDirty, savedPreference, sessionVisits, videoStatus
document.getElementById('fullName').value = 'Test User'
     → Sets the name field value from the console

4. Sources Panel — Add Breakpoints
   Click the Sources tab
   Open index.html → find the handleSubmit function
   Click the line number next to: const name = document.getElementById...
A blue marker appears — this is a breakpoint
   Submit the registration form
   Script pauses at the breakpoint
   Hover over 'name' variable → shows its current value
   Step Over (F10) to advance line by line    Watch panel:
add "name" and "event" to watch expressions
   Resume (F8) to continue execution

5. Network Panel — Check Resources
   Click the Network tab, reload the page
   See all files loaded: index.html, CSS, images, video    Click
on any request to see Headers, Preview, Response, Timing

6. Application Panel — Inspect Storage
   Click the Application tab
```

Expand: Local Storage →
http://localhost preferredEvent:
"tech" Expand: Session Storage
visitCount: "3"

Right-click a key → Delete to remove it manually

Console tab output on page load:

Portal script loaded.

■ Initialisation

Saved preference from localStorage: tech

Session visit count: 3

After typing debugState() in console:

(index)	Value
formDirty	false
savedPreference	"tech"
sessionVisits	"3"
videoStatus	"Video ready to play..."

Breakpoint hit (handleSubmit):

Execution paused at line 12

Variable tooltip: name = "Rahul Sharma"

Call stack shows: handleSubmit onclick

Network panel:

GET index.html 200 1.2 kB 12 ms

GET images/*.jpg 200 45 kB 8 ms each GET videos/*.mp4 206 4.2 MB partial (range request for video streaming)

Elements panel live edit: nav { background-color: #e63946; } (red — only in DevTools, resets on reload) Unchecking "padding: 12px 24px" collapses the nav height immediately